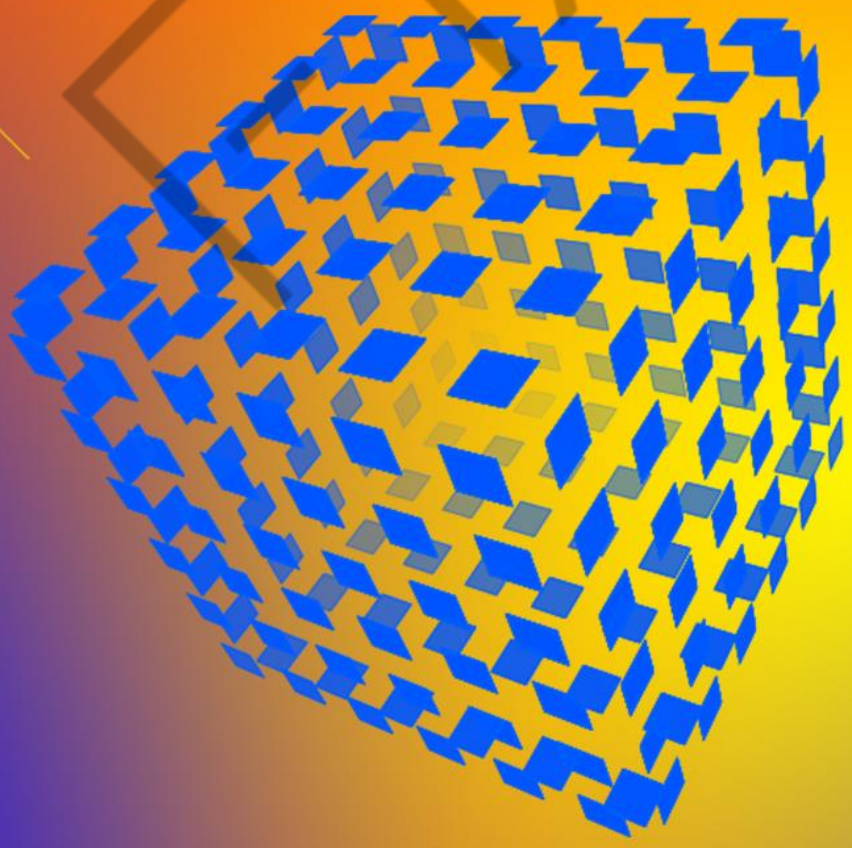
FIAP ON

SAFRA DO CONHECIMENTO
CIBERNÉTICO - PRAGAS VIRTUAIS

CIÊNCIA DE DADOS AO ALCANCE DAS MÃOS COM STREAMLIT



4

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

LISTA DE FIGURAS

Figura 1 - Logo Streamlit 6
Figura 2 - Streamlit hello 8
Figura 3 - Streamlit hello no navegador 8
Figura 4 - VSCode rodando 10
Figura 5 – “Hello, World!” rodando 10
Figura 6 - Forma errada de executar o código 11
Figura 7 - Jogo Quiz 12
Figura 8 - Widgets de entrada 16
Figura 9 - Widgets de iteratividade 19
Figura 10 - Widgets de experiência 21
Figura 11 - Interface da aplicação 30
Figura 12 - Análise descritiva 39
Figura 13 - Modelagem preditiva 44
Figura 14 - Novas predições do modelo 47
Figura 15 - Streamlit Cloud 49
Figura 16 - Publicado no Streamlit Cloud 50

FIAP

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

LISTA DE CÓDIGOS-FONTE

Código-fonte 1 - Exemplo básico 9
Código-fonte 2 - Quiz – título, descrição e perguntas 13
Código-fonte 3 - Quiz – Session_state.....14
Código-fonte 4 - Widgets de entrada.....16
Código-fonte 5 - Widgets de iteratividade 17
Código-fonte 6 - Widgets de experiência 20
Código-fonte 7 - Quiz – exibe perguntas.....23
Código-fonte 8 - Quiz – respostas.....24
Código-fonte 9 - Quiz – CSS botão 25
Código-fonte 10 - Quiz – jogo completo 27
Código-fonte 11 - App.py 31
Código-fonte 12 - data_generator 33
Código-fonte 13 - Exploracao_de_dados.py 39
Código-fonte 14 - Treinamento_de_dados.py 43
Código-fonte 15 - Novas predições.....46

FIAP

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

LISTA DE COMANDOS DE PROMPT DO SISTEMA OPERACIONAL

Comando de prompt 1 - Comando para instalação da biblioteca.....7
Comando de prompt 2 - Comando para instalação da biblioteca usando venv 7
Comando de prompt 3 - Comando para instalação da biblioteca usando conda 7
Comando de prompt 4 - Comando para verificar instalação 8
Comando de prompt 5 - Executa app.py.....9
Comando de prompt 6 - Estrutura do projeto 29
Comando de prompt 7 - Estrutura do projeto revisada.....40

FIAP

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

SUMÁRIO

1 CIÊNCIA DE DADOS AO ALCANCE DAS MÃOS COM STREAMLIT 6
1.1 Introdução 6
1.2 Instalação 7
1.3 Criando o primeiro projeto: Hello, World!.....9
2 CONCEITOS BÁSICOS E INTERATIVIDADE 12
2.1 Jogo: perguntas e respostas 12
2.2 Iniciando a aplicação 12
2.3 Gerenciando o estado da aplicação 14
2.4 Conhecendo widgets para exibir perguntas ao usuário.....15
2.5 Conferindo as respostas.....23
2.6 Estilizando o botão 24
2.7 O jogo completo 26
3 INTEGRAÇÃO CIÊNCIA DE DADOS E MACHINE LEARNING 29
3.1 Organização do projeto e conjunto de dados 29
3.2 Análise exploratória de dados interativa.....34
3.3 Construindo um modelo de Machine Learning 40
3.4 Fazendo previsões com o modelo.....44
4 DEPLOY E COMPARTILHAMENTO.....48
4.1 Compartilhando aplicações com o Streamlit Cloud 48
4.2 Publicando sua aplicação.....49
CONCLUSÃO.....51
REFERÊNCIAS.....52

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

1 CIÊNCIA DE DADOS AO ALCANCE DAS MÃOS COM STREAMLIT

1.1 Introdução

O Streamlit é um *framework* open-source em Python que simplifica o processo de criação de aplicações Web interativas para projetos de Ciência de Dados e Machine Learning. Com o Streamlit, você pode transformar scripts de análise de dados em aplicativos web intuitivos de forma simples em algumas linhas de código, sem a necessidade de conhecimento em desenvolvimento web tradicional.

Figura 1 - Logo Streamlit
Fonte: Streamlit (2024)

Esta biblioteca possui diversas características que a tornam uma boa alternativa para criação de aplicações Web em projetos de Ciência de Dados e Machine Learning, tais como:

- **Simplicidade:** não é necessário entender HTML, CSS ou JavaScript.
- **Rápida prototipação:** desenvolva e visualize resultados em tempo real.
- **Integração fluida:** compatível com bibliotecas populares como Pandas, NumPy, Matplotlib, Scikit-learn, entre outras.
- **Interatividade:** crie facilmente widgets como sliders, botões e caixas de seleção.

É importante destacar que o Streamlit foi criado com o foco no desenvolvimento rápido de aplicativos interativos com menos código e complexidade. Ou seja, ele é útil

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

para prototipagem e aplicações que requerem desenvolvimento ágil, pois o Streamlit abstrai a camada web, permitindo que você se concentre na lógica de negócios e nas visualizações de dados.

Agora que você compreende o que é o Streamlit e suas vantagens, está pronto para colocar a mão na massa. Na próxima seção, vamos abordar a instalação e configuração do ambiente para que você possa criar seu primeiro aplicativo Streamlit.

1.2 Instalação

A instalação do Streamlit é realizada por meio do gerenciador de pacotes *pip* do Python. Para realizar a instalação, abra o terminal ou prompt de comando e execute o seguinte comando:

```
pip install streamlit
```

Comando de prompt 1 - Comando para instalação da biblioteca
Fonte: Elaborado pelo autor (2024)

É importante criar um ambiente virtual para gerenciar as dependências de seus projetos Python. Você pode criar um ambiente virtual utilizando venv ou conda (para usuários Anaconda).

- **Venv:**

```
python -m venv
venv\Scripts\activate  # activate
```

Comando de prompt 2 - Comando para instalação da biblioteca usando venv
Fonte: Elaborado pelo autor (2024)

- **conda:**

```
create --name venv python=3.9
conda activate venv
conda install -c conda-forge streamlit
```

Comando de prompt 3 - Comando para instalação da biblioteca usando conda
Fonte: Elaborado pelo autor (2024)

FIAP

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

Podemos verificar se a instalação está correta executando uma aplicação de demonstração. Para isso, abra o terminal ou prompt de comando e execute o seguinte comando:

```
streamlit hello
```

Comando de prompt 4 - Comando para verificar instalação
Fonte: Elaborado pelo autor (2024)

É esperado como resultado uma mensagem de boas-vindas informando que o serviço foi inicializado com a URL em *localhost* e geralmente na porta 8501. Como ilustra a figura “Streamlit hello”.

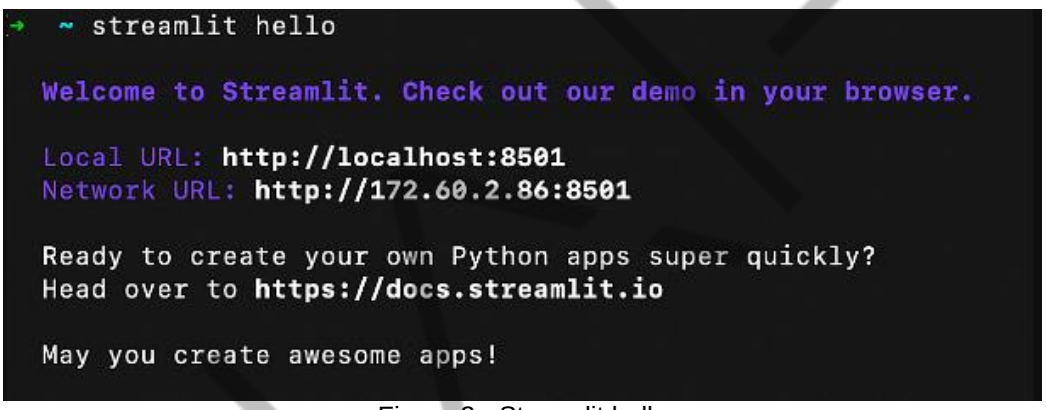


Figura 2 - Streamlit hello
Fonte: Elaborado pelo autor (2024)

Isso abrirá no navegador uma aplicação de demonstração do Streamlit mostrando exemplos de gráficos, mapas e widgets interativos.

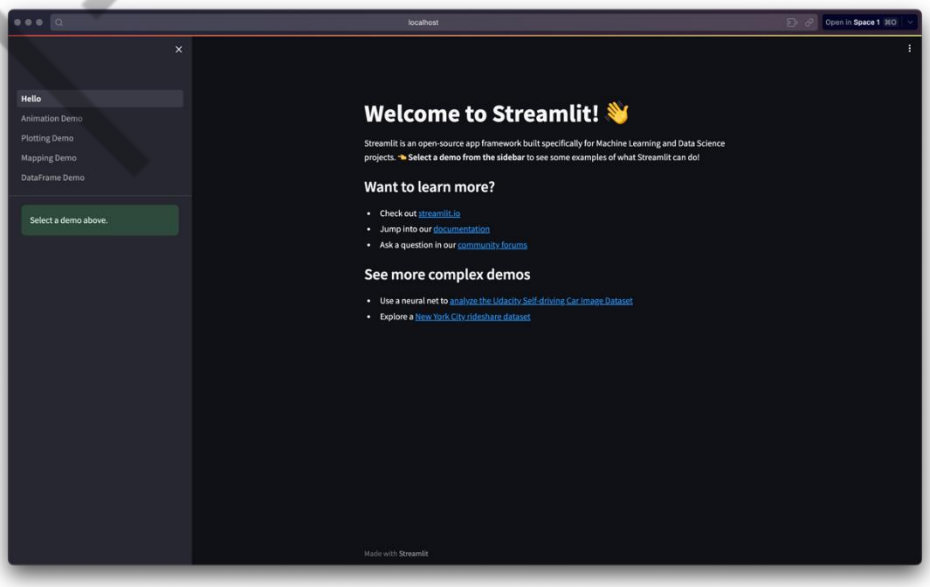


Figura 3 - Streamlit hello no navegador
Fonte: Elaborado pelo autor (2024)

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

Dica: se a aplicação não abrir automaticamente, copie e cole o Local URL exibido no terminal em seu navegador.

1.3 Criando o primeiro projeto: Hello, World!

Com a instalação feita, é hora de fazer o que todo desenvolvedor sabe, que é quase um ritual: rodar o famoso "Hello, World!". Afinal, você só pode dizer que realmente começou a usar uma nova ferramenta depois de ter seu próprio "Hello, World!" rodando. Para isso, crie um arquivo Python, como por exemplo app.py, com o seguinte conteúdo básico:

```
import streamlit as st

st.title('Hello, FIAP!')
st.write('Let`s Rock the Future!')
```

Código-fonte 1 - Exemplo básico
Fonte: Elaborado pelo autor (2024)

No terminal ou cmd, navegue até o diretório onde o arquivo app.py está salvo e rode o seguinte comando:

```
streamlit run app.py
```

Comando de prompt 5 - Executa app.py
Fonte: Elaborado pelo autor (2024)

É esperado que terminal exiba a URL em que o Streamlit está rodando:

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

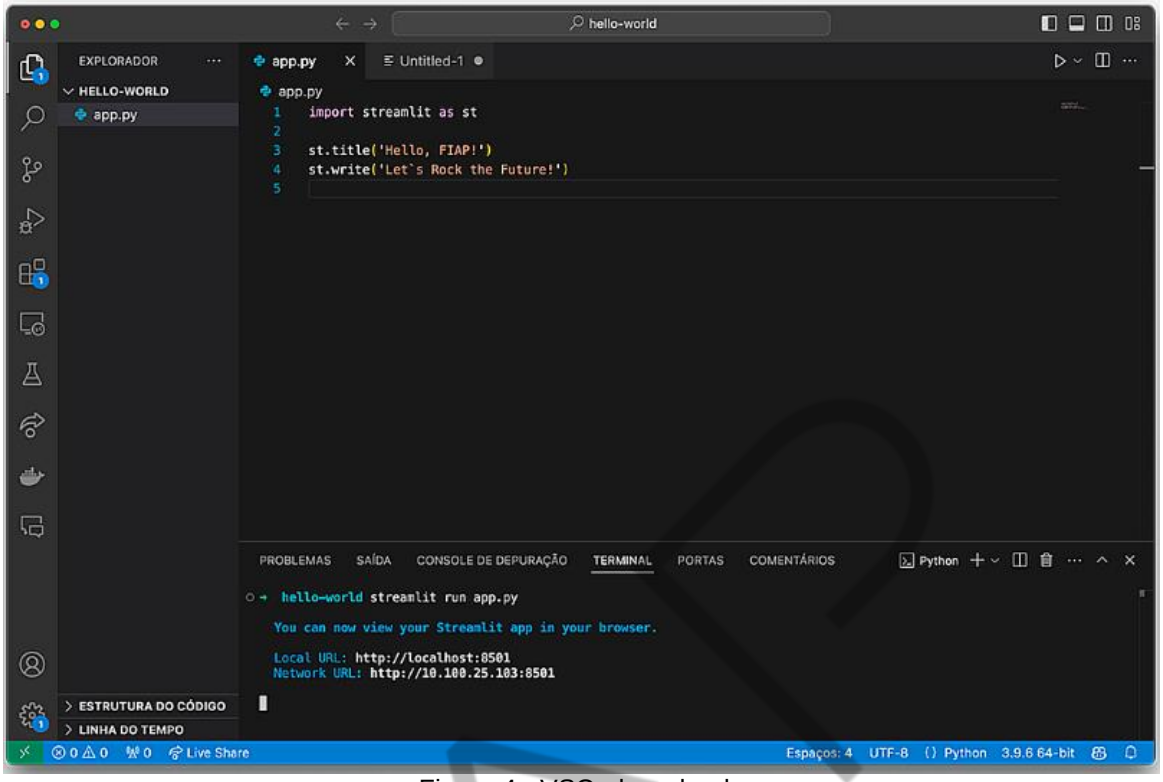


Figura 4 - VSCode rodando
Fonte: Elaborado pelo autor (2024)

Ao abrir no navegador vamos ver o nosso “Hello, World!".

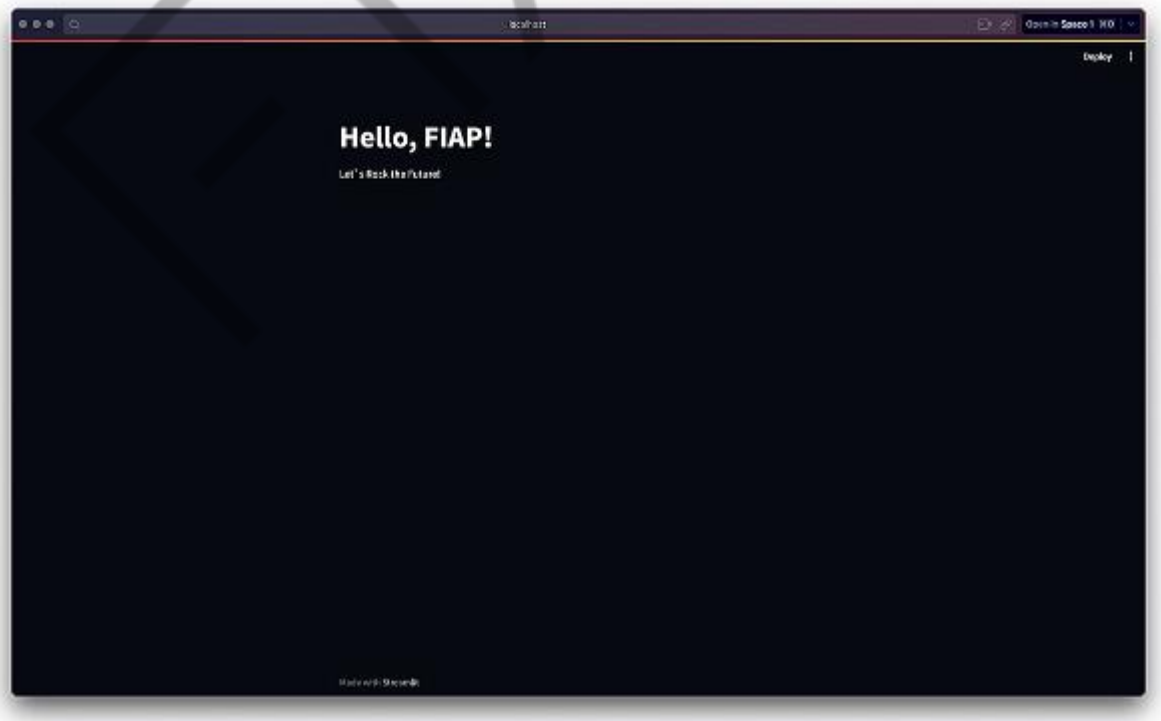


Figura 5 – “Hello, World!” rodando
Fonte: Elaborado pelo autor (2024)

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

Importante:

- Se estiver usando o VSCode, não clique no botão "play" para executar o código.

- Se tentar rodar o arquivo com o botão "play" (ou o atalho de execução padrão), o Streamlit não vai rodar corretamente.

- O Streamlit não é apenas um script Python normal. Ele é uma aplicação web e precisa ser iniciado por um servidor local, que é justamente o que o comando streamlit run faz. O botão "play" do VSCode simplesmente executa o Python diretamente e, nesse caso, isso não é suficiente para rodar a aplicação Streamlit corretamente.

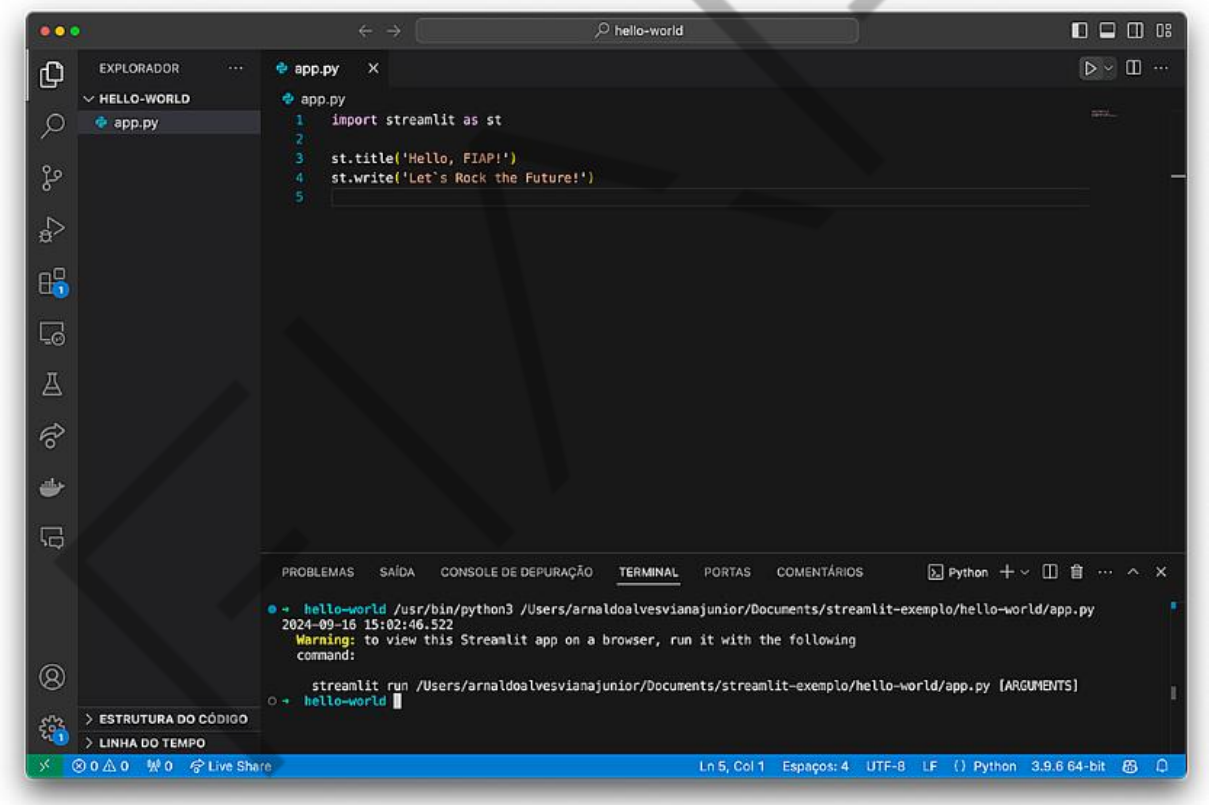


Figura 6 - Forma errada de executar o código
Fonte: Elaborado pelo autor (2024)

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

2 CONCEITOS BÁSICOS E INTERATIVIDADE

Vamos aprender os conceitos fundamentais do Streamlit enquanto desenvolvemos um mini projeto. Em vez de apresentar comandos de forma isolada, vamos desenvolver um jogo simples passo a passo.

2.1 Jogo: perguntas e respostas

O nosso jogo é um `Quiz` ou jogo de perguntas e respostas. O jogo deve apresentar perguntas de múltipla escolha ao usuário e permitir que ele selecione suas respostas. Ao final, deve fornecer feedback imediato sobre as respostas certas e erradas e exibir uma pontuação total. O nosso jogo vai ser parecido com a figura “Jogo Quiz”.

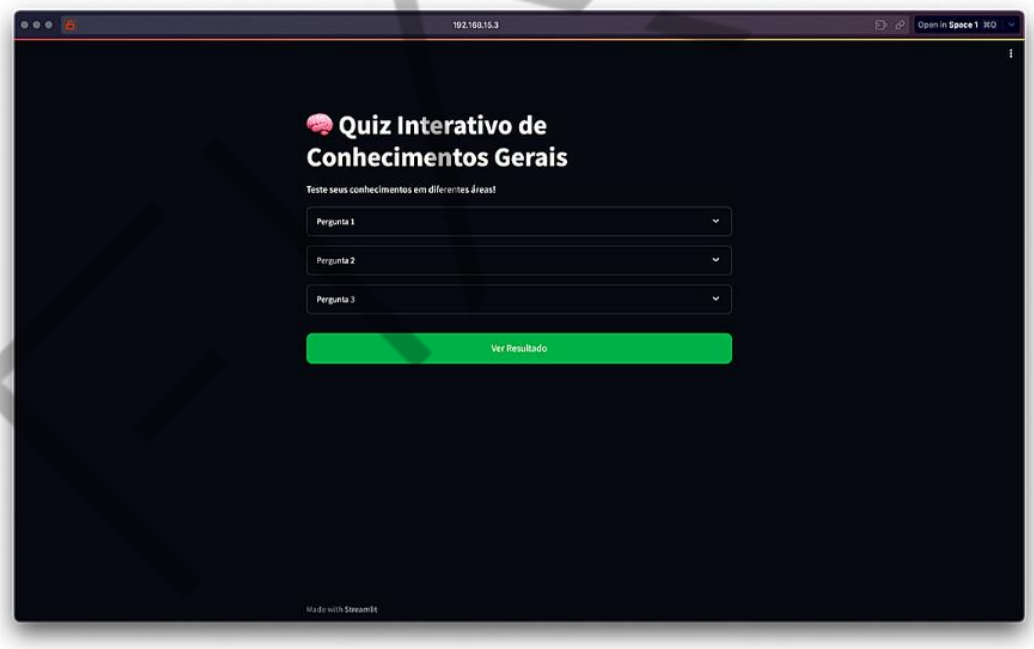


Figura 7 - Jogo Quiz
Fonte: Elaborado pelo autor (2024)

2.2 Iniciando a aplicação

Começamos nosso projeto criando um arquivo chamado “quiz_app.py” em uma pasta de trabalho de sua preferência.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

Vamos importar a biblioteca Streamlit como st e, em seguida, usar os comandos st.title() e st.write() para adicionar um título e uma breve descrição do projeto para o usuário.

Para criar o quiz, precisamos definir as perguntas, opções e respostas corretas. Utilizaremos uma lista de dicionários para organizar essas informações, onde cada dicionário representa uma pergunta, suas opções e a resposta correta:

```
import streamlit as st

st.title(' Quiz Interativo de Conhecimentos Gerais')
st.write(' Teste seus conhecimentos')

# Lista de perguntas
perguntas = [
    {
        "pergunta": "Qual é a capital da França?",
        "opcoes": ["Londres", "Berlim", "Paris", "Roma"],
        "resposta": "Paris"
    },
    {
        "pergunta": "Quem escreveu 'Hamlet'?",
        "opcoes": ["Charles Dickens", "William
Shakespeare", "Mark Twain", "Leo Tolstoy"],
        "resposta": "William Shakespeare"
    },
    {
        "pergunta": "Qual é o elemento químico
representado pelo símbolo 'O'?",
        "opcoes": ["Ouro", "Oxigênio", "Prata", "Ferro"],
        "resposta": "Oxigênio"
    },
    # Adicione mais perguntas conforme desejar
]
```

Código-fonte 2 - Quiz – título, descrição e perguntas
Fonte: Elaborado pelo autor (2024)

Dica: por que usar uma lista de dicionários?

- Facilita a iteração sobre as perguntas.

- Permite adicionar, remover ou modificar perguntas facilmente.

- Organiza as informações de maneira estruturada.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

2.3 Gerenciando o estado da aplicação

Para que nossa aplicação seja interativa e mantenha as escolhas do usuário ao longo do uso, precisamos gerenciar o estado da aplicação.

- **st.session_state:** é um objeto semelhante a um dicionário que permite armazenar informações persistentes entre interações do usuário com a aplicação. Isso é importante porque o Streamlit executa novamente todo o script a cada interação (como clicar em um botão ou selecionar uma opção), e sem o gerenciamento de estado, as informações seriam perdidas a cada atualização.
- Entre as vantagens, podemos destacar os seguintes pontos:
  - Persistência de dados: para armazenar dados que precisam ser mantidos durante toda a sessão do usuário.
  - Gerenciamento de interações: para controlar o fluxo da aplicação com base nas ações do usuário.
  - Armazenamento de variáveis: para guardar valores que não queremos que sejam redefinidos a cada execução do script.

No nosso quiz, queremos armazenar as respostas que o usuário selecionou para cada pergunta. Para isso, vamos inicializar um dicionário dentro do st.session_state que irá armazenar essas respostas. Assim, quando o usuário clicar em "Ver Resultado", teremos todas as respostas armazenadas.

```
    if 'respostas' not in st.session_state:
        st.session_state.respostas = {}
```

Código-fonte 3 - Quiz – Session_state
Fonte: Elaborado pelo autor (2024)

Dica: pense no st.session_state como a “memória” da sua aplicação. Sem ele, toda vez que o usuário interagisse com um elemento, as informações anteriores seriam perdidas. Com o st.session_state, podemos manter informações importantes durante toda a sessão.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

2.4 Conhecendo widgets para exibir perguntas ao usuário

Chegou o momento em que precisamos que nosso jogo mostre as perguntas ao usuário e capture suas respostas. Para isso, vamos utilizar widgets interativos do Streamlit.

Os **widgets** são elementos da interface do usuário que permitem interação, como botões, caixas de seleção, campos de entrada de texto, entre outros. No Streamlit, esses widgets permitem que os usuários interajam com a aplicação de forma dinâmica.

Existem widgets para as mais diversas aplicações e necessidades. Vamos conhecer alguns, dividindo-os em três principais grupos:

1. **Entrada de dados:** permite que o usuário insira informações que serão utilizadas pela aplicação, como nomes, comentários, números e arquivos. Como no trecho de código a seguir:

```
import streamlit as st

st.title('Hello, FIAP!')
st.write('Let`s Rock the Future!')

# widgets de entrada

# Entrada de Texto
nome = st.text_input("Digite seu nome:")
if nome:
    st.write(f"Olá, {nome}!")

# Área de Texto
comentario = st.text_area("Deixe seu comentário:")
if comentario:
    st.write(f"Comentário enviado: {comentario}")

# Entrada Numérica
idade = st.number_input("Informe sua idade:",
min_value=0, max_value=120, value=25)
st.write(f"Idade informada: {idade}")

# Carregamento de Arquivos
arquivo = st.file_uploader("Envie um arquivo de texto:",
type=["txt"])
if arquivo is not None:
```

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

```
        conteudo = arquivo.read().decode("utf-8")
        st.write(conteudo)
```

Código-fonte 4 - Widgets de entrada
Fonte: Elaborado pelo autor (2024)

Ao executar o código, você verá uma interface que permite inserir seu nome, deixar um comentário, informar sua idade e enviar um arquivo de texto. Cada widget está disposto na ordem em que aparece no código.

- **st.text_input:** cria uma caixa de entrada de texto que permite apenas uma linha. Ideal para nomes, endereços de e-mail ou outros dados curtos.
- **st.text_area:** fornece uma área de texto que permite várias linhas. Útil para comentários, descrições ou qualquer entrada de texto mais extensa.
- **st.number_input:** permite a entrada de números, com opções para definir valores mínimos, máximos e um valor padrão.
- **st.file_uploader:** permite que o usuário envie arquivos para a aplicação. Podemos especificar os tipos de arquivos aceitos no argumento type.

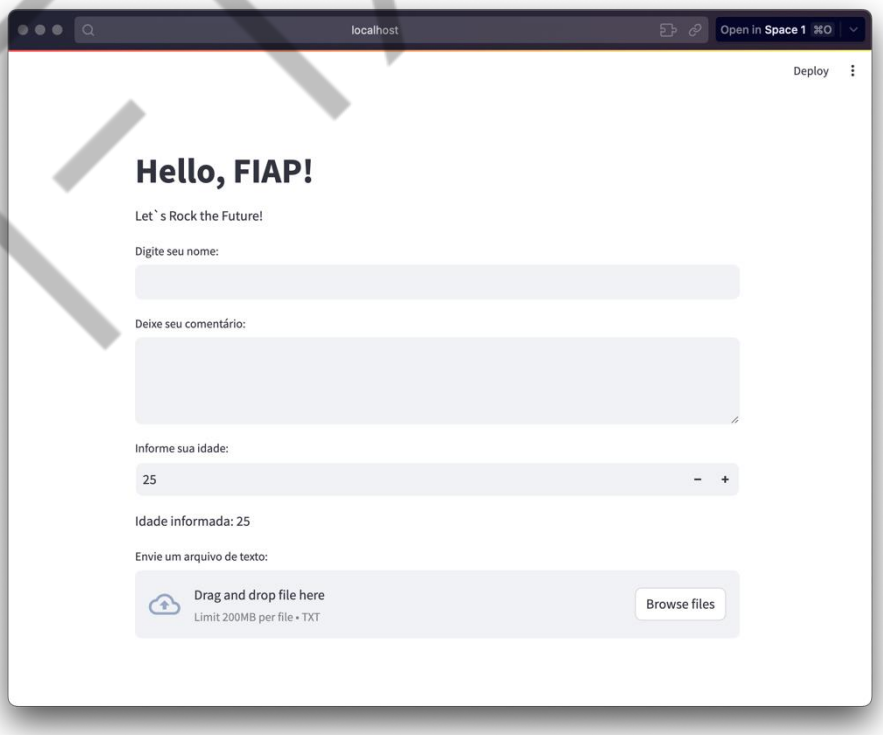


Figura 8 - Widgets de entrada
Fonte: Elaborado pelo autor (2024)

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

2. **Interatividade:** tornam a aplicação dinâmica e responsiva às ações do usuário, permitindo que a aplicação reaja em tempo real. Isso inclui botões, caixas de seleção e outros elementos que capturam ações específicas.

```
import streamlit as st

st.title('Hello, FIAP!')
st.write('Let`s Rock the Future!')

# widgets de iteração

# Botão de Ação
if st.button("Clique aqui"):
    st.write("Você clicou no botão!")

# Caixa de Seleção (checkbox)
aceitar = st.checkbox("Eu aceito os termos e condições.")
if aceitar:
    st.write("Obrigado por aceitar os termos!")

# Botões de Rádio
cor = st.radio("Escolha sua cor favorita:", ["Azul",
"Verde", "Vermelho"])
st.write(f"Você escolheu: {cor}")

# Caixa de Seleção (selectbox)
fruta = st.selectbox("Escolha uma fruta:", ["Maçã",
"Banana", "Laranja"])
st.write(f"Você escolheu: {fruta}")

# Controle Deslizante
altura = st.slider("Escolha sua altura:", min_value=1.0,
max_value=2.5, value=1.75)
st.write(f"Sua altura: {altura} metros")

# Seleção Múltipla
hobbies = st.multiselect("Escolha seus hobbies
favoritos:", ["Ler", "Esportes", "Música", "Viagens"])
st.write(f"Você escolheu: {', '.join(hobbies)}")
```

Código-fonte 5 - Widgets de iteratividade
Fonte: Elaborado pelo autor (2024)

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

Ao interagir com os widgets, observe como a aplicação responde imediatamente às suas ações. Por exemplo, ao clicar no botão ou marcar a caixa de seleção, o conteúdo exibido muda de acordo com a interação.

A escolha do widget ideal para interação dependerá do objetivo da sua aplicação. Não há uma escolha certa ou errada; é recomendável testar diferentes widgets para encontrar o melhor formato que atende às necessidades do usuário:

- **st.button:** cria um botão que, ao ser pressionado, retorna True uma única vez.
- **st.checkbox:** apresenta uma caixa de seleção que pode ser marcada ou desmarcada.
- **st.radio:** permite que o usuário escolha uma opção entre várias, apresentadas como botões de rádio.
- **st.selectbox:** similar ao radio, mas as opções são apresentadas em um menu suspenso.
- **st.slider:** fornece um controle deslizante para selecionar um valor dentro de um intervalo.
- **st.multiselect:** permite que o usuário selecione múltiplas opções de uma lista.

FIAP

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

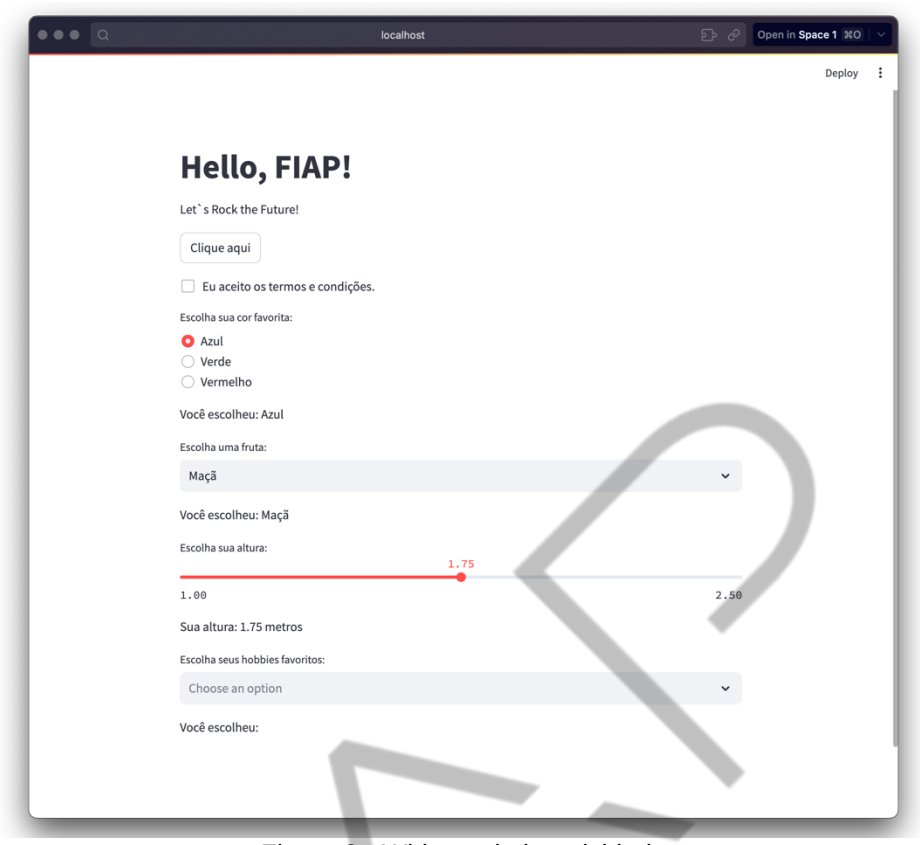


Figura 9 - Widgets de iteratividade
Fonte: Elaborado pelo autor (2024)

3. **Melhoria da experiência:** além dos widgets de entrada e interatividade, o Streamlit oferece elementos que melhoram a experiência do usuário, fornecendo feedback visual e mantendo o usuário informado sobre o estado da aplicação.

```
import streamlit as st

st.title('Hello, FIAP!')
st.write('Let`s Rock the Future!')

# Barra de Progresso
import time

progresso = st.progress(0)
for i in range(100):
    time.sleep(0.02)
    progresso.progress(i + 1)
st.write("Progresso completo!")

# Indicador de Carregamento
with st.spinner("Carregando..."):
    time.sleep(2)
st.success("Carregamento concluído!")
```

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

```
    # Mensagens de feedback visual ao usuário
    st.success("Operação realizada com sucesso!")
    st.error("Ocorreu um erro!")
    st.warning("Atenção! Verifique os dados.")
    st.info("Para mais informações, consulte a documentação.")

    # Animação de Balões
    if st.button("Mostrar balões"):
        st.balloons()
```

Código-fonte 6 - Widgets de experiência
Fonte: Elaborado pelo autor (2024)

Fornecer feedback visual é importante para uma boa experiência do usuário. Isso mantém o usuário informado sobre o que está acontecendo e ajuda a evitar confusões ou a sensação de que a aplicação está travada:

- **st.progress:** exibe uma barra de progresso que pode ser atualizada para refletir o andamento de uma tarefa. Simulamos no código um processo que leva algum tempo, atualizando a barra de progresso a cada iteração do loop.
- **st.spinner:** mostra um indicador de carregamento (*spinner*) durante a execução de um bloco de código.
- **st.info, st.success, st.warning e st.error**: exibem mensagens informativas, de sucesso, de aviso e de erro, respectivamente.
- **st.balloons:** exibe uma animação de balões, geralmente usada para celebrar algum evento ou conclusão de tarefa.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

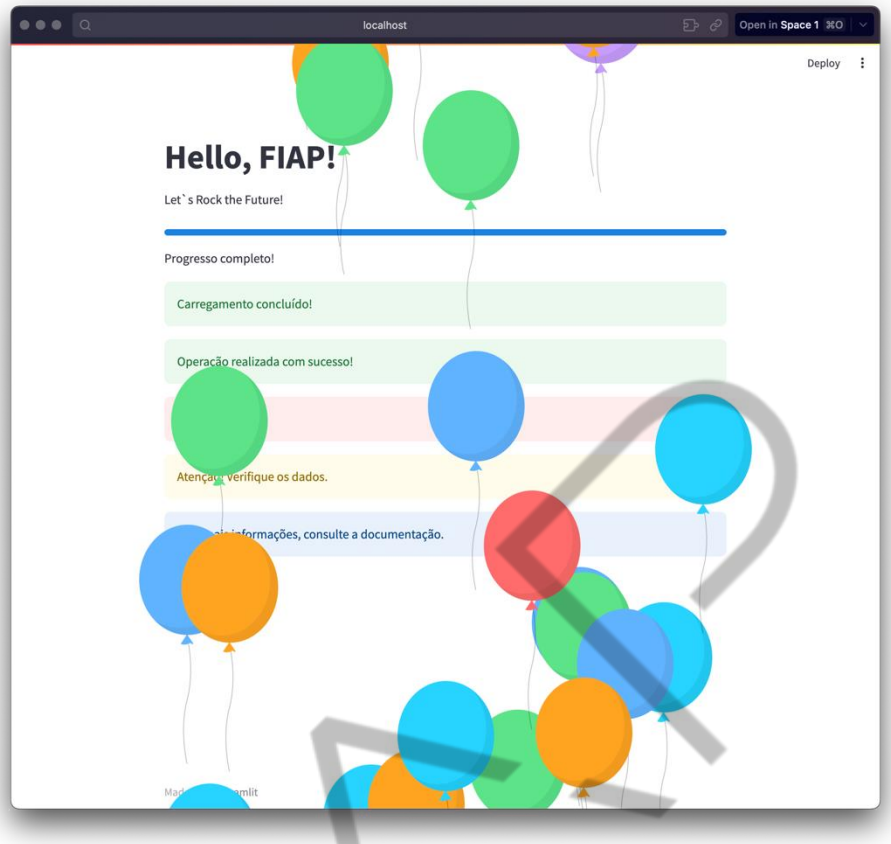


Figura 10 - Widgets de experiência
Fonte: Elaborado pelo autor (2024)

Dica: acesse a documentação oficial e saiba mais sobre esses e outros widgets disponíveis:

API Reference – Widgets: <https://docs.streamlit.io/develop/api-reference/widgets>.

Agora que conhecemos um pouco sobre os principais widgets, vamos voltar ao nosso jogo Quiz. Esta é a parte central (*core*) da nossa aplicação. É aqui que as perguntas são apresentadas ao usuário e as respostas são capturadas. Essa parte do código foi pensada para ser executada de forma interativa, permitindo que o usuário selecione as respostas usando widgets. Por esse motivo, é importante dividir essa parte em alguns passos:

- Percorrer cada pergunta na lista perguntas;
- Criar um expander para cada pergunta, de forma organizada;

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

- Exibir o texto da pergunta e as opções de resposta com st.radio dentro do expande;
- Armazenar a resposta do usuário no st.session_state.

Primeiramente, precisamos percorrer todas as perguntas que criamos. Para isso, utilizamos um “loop for” com a função enumerate que nos dá acesso tanto ao `idx`, que representa o índice, quanto ao `item`, que representa o dicionário de cada pergunta. O enumerate é uma função importante, pois nos ajuda a criar chaves únicas para os widgets, evitando conflitos.

O segundo passo é organizar a nossa interface, especialmente em quizzes com muitas perguntas, para isso vamos permitir que o usuário expanda ou recolha cada pergunta. Desta forma, o usuário consegue focar em uma questão por vez para responder. O widget st.expander cria um contêiner que pode ser expandido ou recolhido.

No contêiner, vamos apresentar o texto da pergunta e as opções de resposta. Acessamos o texto da pergunta no dicionário `item` chave “pergunta”, e para as opções de respostas utilizamos o widget st.radio, que cria um conjunto de botões de seleção exclusivos (botões de rádio). Vale destacar que é necessário um identificador único para cada widget, sem isso, o Streamlit não é capaz de distinguir entre os widgets dentro do loop, levando a comportamentos indesejados, como respostas sendo sobrescritas ou estados confusos, fazemos isso criando a chave única com key = f"pergunta_{idx}".

Por fim, para que seja possível avaliar as respostas posteriormente, precisamos salvá-las. Usamos o st.session_state, um dicionário que persiste os dados durante a sessão do usuário. A opção selecionada pelo usuário é armazenada na variável resposta_usuario, que é salva durante essa seção do usuário.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

```
    # Exibe as perguntas
    for idx, item in enumerate(perguntas):
        with st.expander(f"Pergunta {idx + 1}"):
            st.write(item["pergunta"])
            resposta_usuario = st.radio(
                "Escolha uma opção:",
                item["opcoes"],
                key=f"pergunta_{idx}"
            )
            st.session_state.respostas[f"pergunta_{idx}"]   =
resposta_usuario
```

Código-fonte 7 - Quiz – exibe perguntas
Fonte: Elaborado pelo autor (2024)

Dica: para entender como o st.session_state armazena os dados, podemos debugar o código com: st.write(st.session_state). Não esqueça de remover ou comentar este código na versão final da aplicação para não expor informações desnecessárias ao usuário.

2.5 Conferindo as respostas

Agora que o usuário respondeu às perguntas do quiz, precisamos avaliar as respostas fornecidas e apresentar um feedback adequado. Essa parte é simples: basta criar um botão "Ver Resultado" que, ao clicar, vai varrer a chave ‘resposta’ do dicionário de perguntas e comparar com o st.session_state as respostas armazenadas. Se o usuário acertar todas as perguntas, será exibida uma animação de balões como forma de celebração.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

```
    if st.button("Ver Resultado"):
        acertos = 0
        for idx, item in enumerate(perguntas):
            resposta_correta = item["resposta"]
            resposta_usuario                                =
st.session_state.respostas.get(f"pergunta_{idx}")

            if resposta_usuario == resposta_correta:
                acertos += 1
                st.success(f"✅ Pergunta {idx + 1}:
Correto!")
            else:
                st.error(f"❌ Pergunta {idx + 1}: Incorreto!
A resposta correta é '{resposta_correta}'.")

        st.write(f"**Você acertou {acertos} de
{len(perguntas)} perguntas.**")
        if acertos == len(perguntas):
            st.balloons()
```

Código-fonte 8 - Quiz – respostas
Fonte: Elaborado pelo autor (2024)

2.6 Estilizando o botão

O nosso jogo Quiz já está funcional e conseguimos jogá-lo, mas que tal aprimorar um pouco mais a interface para deixá-la mais atrativa e bonita? Vamos estilizar o botão "Ver Resultado".

Embora o Streamlit não ofereça suporte direto para estilização avançada, podemos utilizar o CSS para customizar a aparência dos componentes. Para isso, injetamos código CSS na nossa aplicação através do método st.markdown("""<style>...</style>""", unsafe_allow_html=True). Vamos criar um bloco de estilo CSS para alterar as propriedades do botão. Aqui está o código que vamos utilizar:

- **Seletor utilizado:** div.stButton > button:first-child
- **background-color:** #4CAF50; define a cor de fundo do botão (tom de verde).
- **color:** white; define a cor do texto como branco para contraste.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

- **height:** 3em; define a altura do botão em unidades relativas ao tamanho da fonte.
- **width:** 100%; faz o botão ocupar toda a largura disponível.
- **border-radius:** 10px; arredonda as bordas do botão.
- **border:** 2px solid #4CAF50; define uma borda sólida de 2 pixels com a mesma cor do fundo.
- **font-size:** 17px; aumenta o tamanho da fonte para 17 pixels.
- **font-weight:** bold; deixa o texto em negrito.

```
st.markdown(
    """
    <style>
    div.stButton > button:first-child {
        background-color: #4CAF50;
        color: white;
        height: 3em;
        width: 100%;
        border-radius:10px;
        border:2px solid #4CAF50;
        font-size:17px;
        font-weight: bold;
    }
    </style>
    """,
    unsafe_allow_html=True
)
```

Código-fonte 9 - Quiz – CSS botão
Fonte: Elaborado pelo autor (2024)

O Streamlit, por padrão, não permite HTML/CSS direto por questões de segurança. Ao definir unsafe_allow_html=True, estamos informando ao Streamlit que confiamos no código inserido.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

2.7 O jogo completo

Agora que passamos por todas as etapas de criação do nosso jogo Quiz, é hora de vê-lo em ação! Vamos executar o jogo completo, que ao final deve ser parecido com o código a seguir:

```
import streamlit as st

st.title("🧠 Quiz Interativo de Conhecimentos Gerais")
st.write("Teste seus conhecimentos em diferentes áreas!")

# Lista de perguntas
perguntas = [
    {
        "pergunta": "Qual é a capital da França?",
        "opcoes": ["Londres", "Berlim", "Paris", "Roma"],
        "resposta": "Paris"
    },
    {
        "pergunta": "Quem escreveu 'Hamlet'?",
        "opcoes": ["Charles Dickens", "William
Shakespeare", "Mark Twain", "Leo Tolstoy"],
        "resposta": "William Shakespeare"
    },
    {
        "pergunta": "Qual é o elemento químico
representado pelo símbolo 'O'?",
        "opcoes": ["Ouro", "Oxigênio", "Prata", "Ferro"],
        "resposta": "Oxigênio"
    },
    # Adicione mais perguntas conforme desejar
]

if 'respostas' not in st.session_state:
    st.session_state.respostas = {}

# Exibe as perguntas
for idx, item in enumerate(perguntas):
    with st.expander(f"Pergunta {idx + 1}"):
        st.write(item["pergunta"])
        resposta_usuario = st.radio(
            "Escolha uma opção:",
            item["opcoes"],
            key=f"pergunta_{idx}"
        )
        st.session_state.respostas[f"pergunta_{idx}"] =
resposta_usuario
```

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

```
    st.markdown(
        """
        <style>
        div.stButton > button:first-child {
            background-color: #4CAF50;
            color: white;
            height: 3em;
            width: 100%;
            border-radius:10px;
            border:2px solid #4CAF50;
            font-size:17px;
            font-weight: bold;
        }
        </style>
        """,
        unsafe_allow_html=True
    )

    if st.button("Ver Resultado"):
        acertos = 0
        for idx, item in enumerate(perguntas):
            resposta_correta = item["resposta"]
            resposta_usuario                                    =
st.session_state.respostas.get(f"pergunta_{idx}")

            if resposta_usuario == resposta_correta:
                acertos += 1
                st.success(f"✅ Pergunta {idx + 1}:
Correto!")
            else:
                st.error(f"❌ Pergunta {idx + 1}: Incorreto!
A resposta correta é '{resposta_correta}'.")

        st.write(f"**Você acertou {acertos} de
{len(perguntas)} perguntas.**")
        if acertos == len(perguntas):
            st.balloons()
```

Código-fonte 10 - Quiz – jogo completo
Fonte: Elaborado pelo autor (2024)

Desenvolvemos uma aplicação Web com a temática de um jogo Quiz interativo utilizando o Streamlit, aplicamos diversos conceitos importantes dessa biblioteca. Ao longo do processo, aprendemos:

- Os conceitos e comandos básicos da biblioteca.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

- Como criar uma interface interativa com widgets.
- Gerenciar o estado da aplicação para armazenar e recuperar informações do usuário.
- Personalizar a aparência da sua aplicação com CSS.

Exploramos os conceitos e comandos básicos que a biblioteca oferece. O Streamlit possui mais recursos além dos que vimos e é muito aplicado em projetos de Ciência de Dados e Machine Learning, onde a integração é simples com bibliotecas como Pandas, NumPy, Matplotlib, Scikit-learn, TensorFlow e muitas outras.

FIAP

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

3 INTEGRAÇÃO CIÊNCIA DE DADOS E MACHINE LEARNING

Vamos desenvolver um projeto que envolve Ciência de Dados e Machine Learning no contexto do Agronegócio. O objetivo é criar uma aplicação interativa que permita explorar dados agrícolas, realizar análises exploratórias e construir um modelo de Machine Learning para prever a produção agrícola com base em variáveis relevantes.

3.1 Organização do projeto e conjunto de dados

Para organizar projeto de forma eficiente e modular, vamos utilizar o recurso de múltiplas páginas do Streamlit para estruturar melhor o aplicativo e criar a seguinte estrutura básica de pastas e arquivos:

```
projeto_agronegocio/
├── app.py
├── data_generation.py
└── pages/
    ├── 1_Exploracao_de_Dados.py
    └── outras páginas...
```

Comando de prompt 6 - Estrutura do projeto
Fonte: Elaborado pelo autor (2024)

Descrição de cada pasta e arquivo:

- **app.py:** arquivo principal da nossa aplicação Streamlit. Definimos a configuração geral do aplicativo, como o título, o ícone e a página inicial.
- **data_generation.py:** módulo que contém a função generate_data(n_samples) responsável por gerar o dataset simulado.
- **pages/:** pasta que contém os arquivos para cada página da nossa aplicação. O Streamlit reconhece essa pasta e cria automaticamente um menu de navegação com base nos arquivos dentro dela.
- **1_Exploracao_de_Dados.py:** página para exploração de dados. Esta página contém todo o código para realizar visualizações interativas e análises descritivas do dataset.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

- Outras páginas podem ser adicionadas ao projeto criando novos arquivos Python.

Dica: o prefixo numérico 1_ no nome do arquivo dentro da pasta pages/ determina a ordem em que a página aparece no menu de navegação do Streamlit. Se adicionar mais páginas, utilize o prefixo numérico 2_, 3_ e assim por diante.

Após a criação da estrutura de pastas do projeto, começamos o desenvolvimento da nossa aplicação pela página principal. Ela será uma página simples sem informações técnicas, apenas apresentando a ideia principal da nossa aplicação. O usuário irá navegar em nosso sistema por meio de um menu lateral com as páginas secundárias da aplicação, como demonstra a figura “Interface da aplicação”.

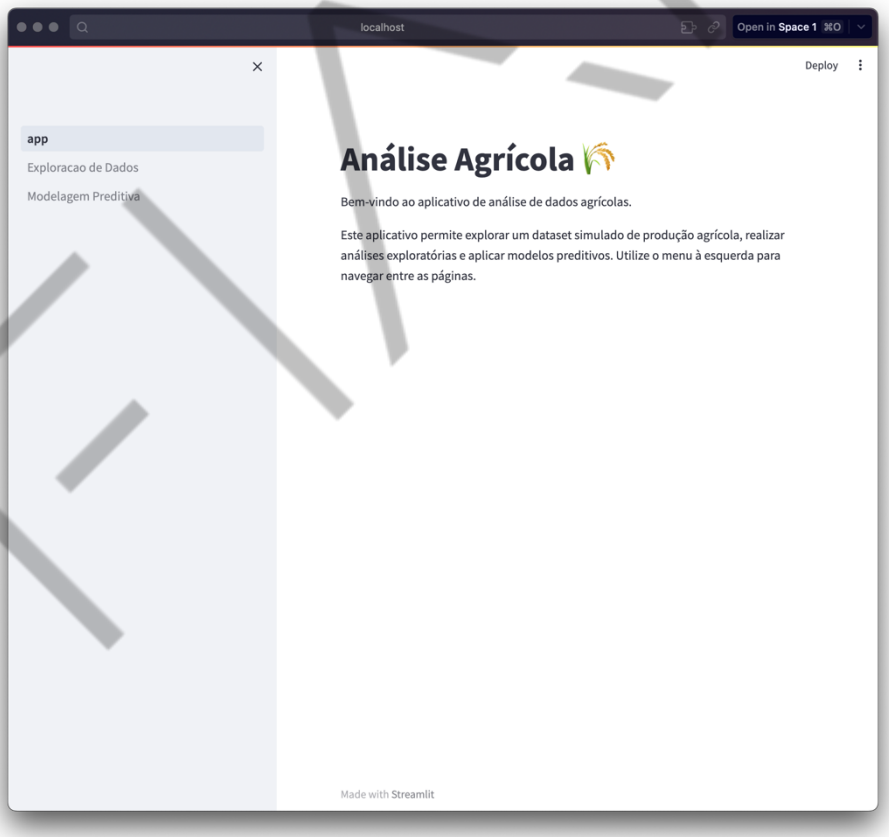


Figura 11 - Interface da aplicação
Fonte: Elaborado pelo autor (2024)

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

No arquivo app.py vamos adicionar o título e descrição do projeto. Note que o código é simples e inicialmente não faz relacionamento com outras páginas, mas o Streamlit reconhece pasta pages e cria automaticamente um menu de navegação com base nos arquivos dentro dela:

```
# app.py

import streamlit as st

st.set_page_config(
    page_title='Análise Agrícola',
    page_icon='🌾',
    layout='wide'
)

st.title('Análise Agrícola 🌾')
st.write('Bem-vindo ao aplicativo de análise de dados agrícolas.')

st.markdown("""
Este aplicativo permite explorar um dataset simulado de produção agrícola,
realizar análises exploratórias e aplicar modelos preditivos.
Utilize o menu à esquerda para navegar entre as páginas.
""")
```

Código-fonte 11 - App.py
Fonte: Elaborado pelo autor (2024)

O Agronegócio desempenha um papel importante na economia global, sendo responsável pela produção de alimentos. Vamos criar um conjunto de dados simulado que contém informações sobre:

- **Características climáticas:** temperatura média, precipitação, umidade.
- **Práticas agrícolas:** uso de fertilizantes, tipo de solo, técnicas de cultivo.
- **Produção:** rendimento em toneladas por hectare.

O arquivo data_generation.py contém a função generate_data() que cria o nosso dataset com uma quantidade padrão de 1000 dados:

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

```
# data_generation.py

import numpy as np
import pandas as pd
import streamlit as st

@st.cache_data(show_spinner=True) ## usando cache
def generate_data(n_samples=1000):
    """
    Gera um dataset simulado de produção agrícola.

    Args:
        n_samples (int): Número de amostras a serem
geradas.

    Returns:
        pd.DataFrame: DataFrame contendo o dataset
simulado.
    """
    np.random.seed(42)

    # Gerando variáveis climáticas com distribuições
realistas
    temperatura = np.random.normal(loc=22, scale=3,
size=n_samples) # Temperatura média de 22°C
    precipitacao = np.random.gamma(shape=2, scale=30,
size=n_samples) # Precipitação em mm
    umidade = np.random.uniform(low=50, high=80,
size=n_samples) # Umidade relativa em %

    # Variáveis categóricas
    fertilizante = np.random.choice(['Orgânico',
'Sintético'], size=n_samples, p=[0.4, 0.6])
    tipo_solo = np.random.choice(['Arenoso', 'Argiloso',
'Siltoso'], size=n_samples, p=[0.3, 0.5, 0.2])

    # Mapear categorias para valores numéricos (efeitos)
    mapa_fertilizante = {'Orgânico': 1.05, 'Sintético':
1.10}
    mapa_solo = {'Arenoso': 0.95, 'Argiloso': 1.00,
'Siltoso': 1.02}

    efeito_fertilizante =
np.vectorize(mapa_fertilizante.get)(fertilizante)
    efeito_solo = np.vectorize(mapa_solo.get)(tipo_solo)

    # Calculando a produção com interações mais realistas
    producao = (
        (temperatura - 20) * 2 + # Aumento de produção
por temperatura acima de 20°C
```

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

```
            (precipitacao - 50) * 0.5 + # Aumento por
precipitação acima de 50mm
            (umidade - 50) * 0.3  # Aumento por umidade acima
de 50%
        ) * efeito_fertilizante * efeito_solo

        # Adicionando variabilidade aleatória
        producao += np.random.normal(loc=0, scale=5,
size=n_samples)

        # Garantindo que a produção não seja negativa
        producao = np.clip(producao, a_min=0, a_max=None)

        # Criando o DataFrame final
        df = pd.DataFrame({
            'Temperatura': temperatura,
            'Precipitação': precipitacao,
            'Umidade': umidade,
            'Fertilizante': fertilizante,
            'Tipo de Solo': tipo_solo,
            'Produção': producao
        })

        return df
```

Código-fonte 12 - data_generator
Fonte: Elaborado pelo autor (2024)

Para a criação do dataset, cada atributo foi definido de acordo com as seguintes premissas:

- **Temperatura:** média de 22 °C com desvio padrão de 3 °C, refletindo condições climáticas mais típicas.
- **Precipitação:** usamos uma distribuição gamma para modelar a precipitação, que é sempre positiva e pode representar eventos de chuva.
- **Umidade:** valores entre 50% e 80%, que são comuns em áreas agrícolas.
- **Fertilizante:** atribuímos um aumento de 5% na produção para 'Orgânico' e 10% para 'Sintético', refletindo sua eficácia.
- **Tipo de solo:** o solo 'Arenoso' diminui a produção em 5% devido à menor retenção de água, o 'Argiloso' é neutro, e o 'Siltoso' aumenta em 2% pela melhor fertilidade.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

- **Produção:** consideramos que a produção aumenta linearmente com a temperatura acima de 20 °C, precipitação acima de 50 mm e umidade acima de 50%. Multiplicamos pelo efeito dos fertilizantes e do solo para incorporar suas influências. E por fim, adicionamos um termo aleatório para simular variabilidade não explicada.

Os parâmetros definidos não são necessariamente rígidos, nesse sentido, podem ser alterados conforme a necessidade desejada.

3.2 Análise exploratória de dados interativa

Vamos criar visualizações interativas que permitam ao usuário explorar os dados de diferentes perspectivas. A etapa de análise de dados é importante para compreender os dados que estamos trabalhando. No Python, diversas bibliotecas podem nos ajudar como o Matplotlib, Seaborn, Pandas, Plotly entre outras, todas elas possuem uma integração simples com o Streamlit, sendo possível exibir tabelas e gráficos estáticos e dinâmicos.

Para exibir tabelas de dados no Streamlit, utilizamos a função st.dataframe(). Essa função permite mostrar um DataFrame do Pandas, em que o usuário pode interagir com a tabela, ordenando e explorando os dados. Ela é ideal para apresentar uma visão geral dos dados ou resultados de análises.

Podemos exibir gráficos de algumas formas, entre elas temos:

- Utilizando a função st.pyplot() para exibir gráficos estáticos criados com Matplotlib ou Seaborn;
- Com a função st.plotly_chart() para exibir gráficos interativos criados com Plotly.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

```
# Exploracao_de_dados.py

import streamlit as st
import seaborn as sns
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import plotly.express as px

from data_generation import generate_data   # Carrega o
dataset

# Configurações gerais
st.set_page_config(page_title='Análise     de     Dados',
layout='wide')

# Título do aplicativo
st.title('Exploração de Dados')

# ================================================
# 1. Carregar os Dados
# ================================================

st.header('1. Carregar os Dados')

# Carregar os dados gerados
df = generate_data()

# Mostrar os primeiros registros
st.subheader('Visualização dos Dados')
st.dataframe(df.head())

# ================================================
# 2. Visão Geral dos Dados
# ================================================

st.header('2. Visão Geral dos Dados')

# Informações sobre o DataFrame
st.subheader('Informações do DataFrame')
st.write('Dimensões do DataFrame:')
st.write(f'Linhas:        {df.shape[0]},        Colunas:
{df.shape[1]}')

st.subheader('Tipos de Dados')
# Converter os tipos de dados para string
df_types = pd.DataFrame({
    'Coluna': df.columns,
    'Tipos de Dados': df.dtypes.astype(str)
})
st.write(df_types)
```

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

```
    # Verificar valores ausentes
    st.subheader('Valores Ausentes')
    st.write(df.isnull().sum())

    # Estatísticas Descritivas
    st.subheader('Estatísticas Descritivas')
    st.write(df.describe())

    # ==================================================
    # 3. Análise Univariada
    # ==================================================

    st.header('3. Análise Univariada')

    numeric_columns    =    ['Temperatura',    'Precipitação',
'Umidade', 'Produção']
    categorical_columns = ['Fertilizante', 'Tipo de Solo']

    # Histogramas das variáveis numéricas
    st.subheader('Distribuições das Variáveis Numéricas')
    for col in numeric_columns:
        st.write(f'**{col}**')
        fig    =    px.histogram(df,    x=col,    nbins=30,
title=f'Distribuição de {col}')
        st.plotly_chart(fig, use_container_width=True)

    # Box plots das variáveis numéricas
    st.subheader('Box Plots das Variáveis Numéricas')
    for col in numeric_columns:
        st.write(f'**{col}**')
        fig  =  px.box(df,  y=col,  points='all',  title=f'Box
Plot de {col}')
        st.plotly_chart(fig, use_container_width=True)

    # Distribuição das variáveis categóricas
    st.subheader('Distribuições das Variáveis Categóricas')
    for col in categorical_columns:
        st.write(f'**{col}**')
        fig = px.histogram(df, x=col, title=f'Distribuição de
{col}')
        st.plotly_chart(fig, use_container_width=True)

    # ==================================================
    # 4. Análise Bivariada
    # ==================================================

    st.header('4. Análise Bivariada')

    # Gráficos de dispersão entre variáveis numéricas
```

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

```
    st.subheader('Gráficos de Dispersão entre Variáveis Numéricas')
    variable_pairs = [
        ('Temperatura', 'Produção'),
        ('Precipitação', 'Produção'),
        ('Umidade', 'Produção'),
        ('Temperatura', 'Precipitação'),
        ('Temperatura', 'Umidade')
    ]

    for x_var, y_var in variable_pairs:
        st.write(f'**{x_var} vs {y_var}**')
        fig = px.scatter(
            df,
            x=x_var,
            y=y_var,
            color='Fertilizante',
            symbol='Tipo de Solo',
            title=f'{y_var} vs {x_var}'
        )
        st.plotly_chart(fig, use_container_width=True)

    # Distribuição de Produção por Fertilizante e Tipo de Solo
    st.subheader('Distribuição de Produção por Fertilizante e Tipo de Solo')

    # Por Fertilizante
    st.write('**Produção por Fertilizante**')
    fig = px.violin(
        df,
        x='Fertilizante',
        y='Produção',
        box=True,
        points='all',
        title='Distribuição de Produção por Fertilizante'
    )
    st.plotly_chart(fig, use_container_width=True)

    # Por Tipo de Solo
    st.write('**Produção por Tipo de Solo**')
    fig = px.violin(
        df,
        x='Tipo de Solo',
        y='Produção',
        box=True,
        points='all',
        title='Distribuição de Produção por Tipo de Solo'
    )
    st.plotly_chart(fig, use_container_width=True)

    # ==================================================
```

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

```
    # 5. Análise de Correlação
    # ==================================================

    st.header('5. Análise de Correlação')

    # Mapa de calor de correlação
    st.subheader('Mapa de Calor de Correlação')
    # Selecionar apenas colunas numéricas
    numeric_df = df.select_dtypes(include=[np.number])
    corr = numeric_df.corr()

    fig_heatmap, ax = plt.subplots(figsize=(10, 8))
    sns.heatmap(corr, annot=True, cmap='coolwarm', ax=ax)
    st.pyplot(fig_heatmap)

    # ==================================================
    # 6. Análise Multivariada
    # ==================================================

    st.header('6. Análise Multivariada')

    # Pairplot das variáveis numéricas
    st.subheader('Pair Plot das Variáveis Numéricas')

    # Converter 'Fertilizante' e 'Tipo de Solo' em códigos
numéricos para coloração
    df_encoded = df.copy()
    df_encoded['Fertilizante']                                =
df_encoded['Fertilizante'].map({'Orgânico':  0,  'Sintético':
1})
    df_encoded['Tipo  de  Solo']  =  df_encoded['Tipo  de
Solo'].map({'Arenoso': 0, 'Argiloso': 1, 'Siltoso': 2})

    fig = sns.pairplot(
        df_encoded,
        vars=numeric_columns,
        hue='Fertilizante',
        diag_kind='kde',
        corner=True
    )
    st.pyplot(fig)

    # ==================================================
    # 7. Análise Interativa
    # ==================================================

    st.header('7. Análise Interativa')

    # Seleção de variáveis pelo usuário
    st.subheader('Gráfico de Dispersão Interativo')
```

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

```
    col1, col2, col3 = st.columns(3)

    with col1:
        x_var = st.selectbox('Selecione a variável X:',
options=numeric_columns)
    with col2:
        y_var = st.selectbox('Selecione a variável Y:',
options=numeric_columns)
    with col3:
        color_var = st.selectbox('Selecione a variável para
colorir:', options=categorical_columns)

    corr_temp_prod = df[x_var].corr(df[y_var])
    st.write(f"A correlação de Pearson entre {y_var} e {x_var}
é: {corr_temp_prod:.2f}")

    # Gráfico de dispersão interativo
    fig = px.scatter(
        df,
        x=x_var,
        y=y_var,
        color=color_var,
        title=f'{y_var} vs {x_var} colorido por {color_var}'
    )
    st.plotly_chart(fig, use_container_width=True)
```

Código-fonte 13 - Exploracao_de_dados.py
Fonte: Elaborado pelo autor (2024)

Como resultado, é esperado que a página carregue o dataset e apresente os dados em tabelas e gráficos para apoiar as análises exploratórias de dados, como mostra a figura “Análise descritiva”.

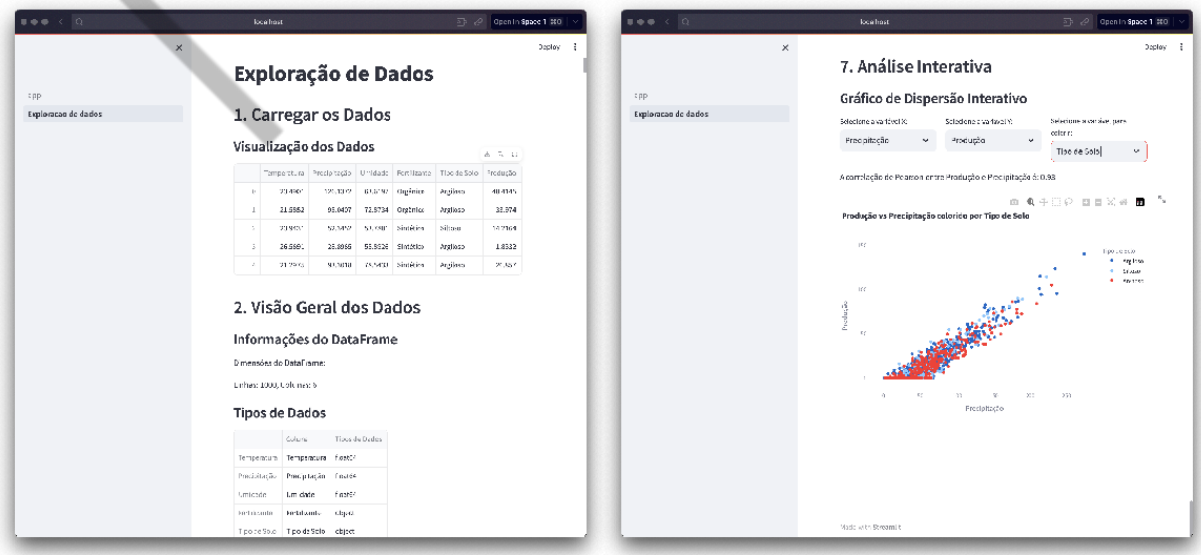


Figura 12 - Análise descritiva
Fonte: Elaborado pelo autor (2024)

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

3.3 Construindo um modelo de Machine Learning

Vamos construir um modelo de regressão para prever a produção com base nas variáveis disponíveis. O Streamlit consegue trabalhar muito bem com bibliotecas de Machine Learning, como Scikit-learn e TensorFlow.

Para manter o nosso projeto organizado e modular, criaremos uma nova página chamada 2_Modelagem_Preditiva.py. A nossa estrutura de arquivos do projeto ficará da seguinte forma:

```
projeto_agronegocio/
├── app.py
├── data_generation.py
└── pages/
    ├── 1_Exploracao_de_Dados.py
    └── 2_Modelagem_Preditiva.py
```

Comando de prompt 7 - Estrutura do projeto revisada
Fonte: Elaborado pelo autor (2024)

Para permitir que o usuário personalize o treinamento do modelo com as features e dados que ele escolher, é necessário implementar filtros interativos na barra lateral da aplicação usando as funções st.sidebar.multiselect() e st.sidebar.slider(), dessa forma filtramos o DataFrame com base nas seleções do usuário.

Podemos aplicar diversas técnicas de pré-processamento de dados, mas o nosso objetivo é compreender as etapas para o treinamento do modelo. Por essa razão, vamos apenas transformar as variáveis categóricas em variáveis numéricas utilizando one-hot encoding com a função pd.get_dummies(). Em seguida, separaremos os dados em treinamento e teste, que permite avaliar a capacidade de generalização do modelo, utilizando: o Random Forest Regressor da Scikit-learn para treinar o modelo; e a métrica de avaliação R2-Score para indicar o quão bem o modelo explica a variabilidade dos dados.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

```
# 2_Modelagem_Preditiva.py

import streamlit as st
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestRegressor

from data_generation import generate_data   # Carrega o
dataset

# Configurações gerais
st.set_page_config(page_title='Modelagem      Preditiva',
layout='wide')

# Título do aplicativo
st.title('Modelagem Preditiva')

# ================================================
# 1. Carregar e Filtrar os Dados
# ================================================

st.header('1. Carregar e Filtrar os Dados')

# Carregar os dados gerados
df = generate_data()

# ----------------------------------
# 1.1. Filtros na Barra Lateral
# ----------------------------------

st.sidebar.title('Filtros de Dados')

# Filtro para Fertilizante
selected_fertilizer = st.sidebar.multiselect(
    'Selecione o Fertilizante:',
    options=df['Fertilizante'].unique(),
    default=df['Fertilizante'].unique()
)

# Filtro para Tipo de Solo
selected_soil = st.sidebar.multiselect(
    'Selecione o Tipo de Solo:',
    options=df['Tipo de Solo'].unique(),
    default=df['Tipo de Solo'].unique()
)

# Filtros para variáveis numéricas

st.sidebar.subheader('Intervalos      das      Variáveis
Numéricas')
```

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

```
    # Temperatura
    temp_min, temp_max = st.sidebar.slider(
        'Temperatura (°C):',
        min_value=float(df['Temperatura'].min()),
        max_value=float(df['Temperatura'].max()),
        value=(float(df['Temperatura'].min()),
float(df['Temperatura'].max()))
    )

    # Precipitação
    precip_min, precip_max = st.sidebar.slider(
        'Precipitação (mm):',
        min_value=float(df['Precipitação'].min()),
        max_value=float(df['Precipitação'].max()),
        value=(float(df['Precipitação'].min()),
float(df['Precipitação'].max()))
    )

    # Umidade
    umid_min, umid_max = st.sidebar.slider(
        'Umidade (%):',
        min_value=float(df['Umidade'].min()),
        max_value=float(df['Umidade'].max()),
        value=(float(df['Umidade'].min()),
float(df['Umidade'].max()))
    )

    # ----------------------------------
    # 1.2. Aplicar Filtros aos Dados
    # ----------------------------------

    # Aplicar os filtros selecionados ao DataFrame
    df_filtered = df[
        (df['Fertilizante'].isin(selected_fertilizer)) &
        (df['Tipo de Solo'].isin(selected_soil)) &
        (df['Temperatura'] >= temp_min) & (df['Temperatura']
<= temp_max) &
        (df['Precipitação'] >= precip_min) &
(df['Precipitação'] <= precip_max) &
        (df['Umidade'] >= umid_min) & (df['Umidade'] <=
umid_max)
    ]

    # Verificar se o DataFrame filtrado não está vazio
    if df_filtered.empty:
        st.warning('Nenhum dado corresponde aos filtros
selecionados. Por favor, ajuste os filtros.')
        st.stop()
    else:
        st.subheader('Dados Filtrados')
        st.dataframe(df_filtered.head())
```

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

```
# ==================================================
# 2. Preparar os Dados para Modelagem
# ==================================================

st.header('2. Preparar os Dados para Modelagem')

# Transformar variáveis categóricas em variáveis dummies
df_ml = pd.get_dummies(df_filtered,
columns=['Fertilizante', 'Tipo de Solo'])

# Separar as variáveis independentes (X) e a variável
dependente (y)
X = df_ml.drop('Produção', axis=1)
y = df_ml['Produção']

# Verificar se há dados suficientes para treinar o modelo
if len(X) < 2:
    st.warning('Dados insuficientes para treinar o
modelo. Por favor, ajuste os filtros para incluir mais dados.')
    st.stop()
else:
    st.write(f'**Total de registros após filtragem:**
{len(X)}')

# ==================================================
# 3. Treinar o Modelo de Machine Learning
# ==================================================

st.header('3. Treinar o Modelo de Machine Learning')

# Dividir os dados em conjuntos de treinamento e teste
X_train, X_test, y_train, y_test = train_test_split(X, y,
test_size=0.2)

# Instanciar o modelo de Random Forest Regressor
model = RandomForestRegressor()

# Treinar o modelo com os dados de treinamento
model.fit(X_train, y_train)

# Avaliar o modelo com os dados de teste
score = model.score(X_test, y_test)
st.write(f'**Acurácia do modelo (R² no conjunto de
teste):** {score:.2f}')
```

Código-fonte 14 - Treinamento_de_dados.py
Fonte: Elaborado pelo autor (2024)

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

Como resultado, é esperado que a nossa aplicação exiba a quantidade total de registros para treinamento com base no filtro definido. Treine o modelo e exiba o R2-Score:

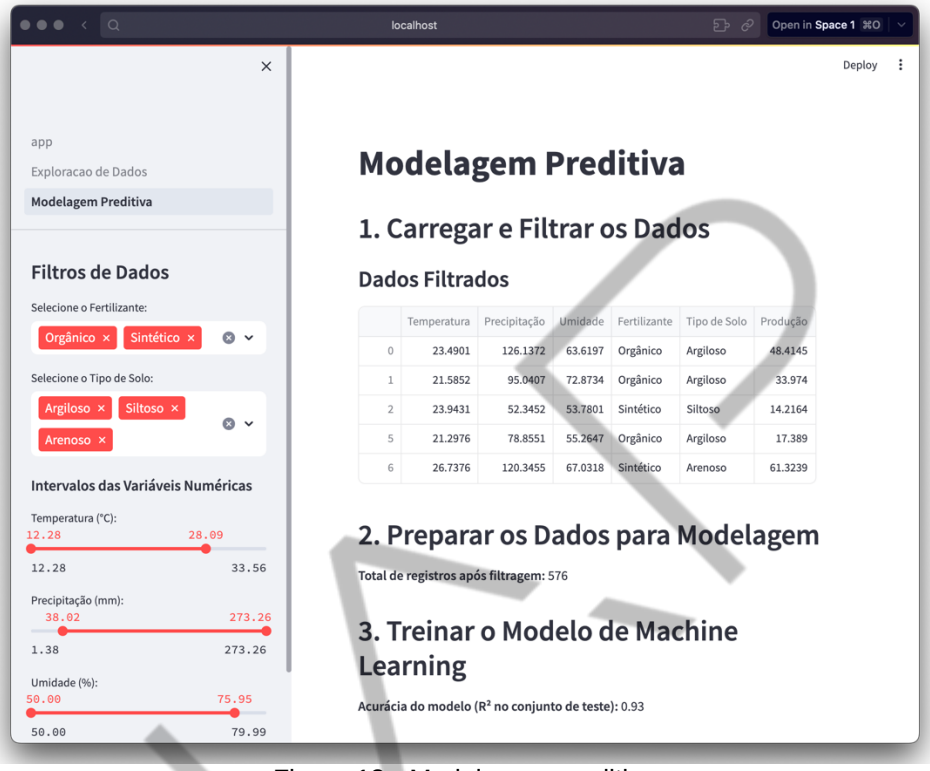


Figura 13 - Modelagem preditiva
Fonte: Elaborado pelo autor (2024)

3.4 Fazendo previsões com o modelo

Com o modelo treinado, podemos dar sequência no arquivo 2_Modelagem_Preditiva.py e criar uma interface para que o usuário possa inserir novos dados e obter previsões:

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

```
    # ==================================================
    # 4. Fazer Previsões com o Modelo
    # ==================================================

    st.header('4. Fazer Previsões com o Modelo')

    st.subheader('Insira os Dados para Previsão')

    # Coletar entrada do usuário para previsão
    temp_input = st.number_input('Temperatura (°C)',
value=float(df['Temperatura'].mean()))
    precip_input = st.number_input('Precipitação (mm)',
value=float(df['Precipitação'].mean()))
    umidade_input = st.number_input('Umidade (%)',
value=float(df['Umidade'].mean()))
    fertilizante_input = st.selectbox('Fertilizante',
df['Fertilizante'].unique())
    solo_input = st.selectbox('Tipo de Solo', df['Tipo de
Solo'].unique())

    # ------------------------------------
    # 4.1. Validar Entradas do Usuário
    # ------------------------------------

    # Inicializar variável de controle
    input_error = False

    # Validar temperatura
    if not (-10 <= temp_input <= 50):
        st.error('A temperatura deve estar entre -10°C e
50°C.')
        input_error = True

    # Validar precipitação
    if not (0 <= precip_input <= 500):
        st.error('A precipitação deve ser entre 0 mm e 500
mm.')
        input_error = True

    # Validar umidade
    if not (0 <= umidade_input <= 100):
        st.error('A umidade deve ser entre 0% e 100%.')
        input_error = True

    # Se não houver erros nas entradas, proceder com a
previsão
    if not input_error:
        # ------------------------------------
        # 4.2. Preparar os Dados de Entrada
        # ------------------------------------
```

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

```
        # Criar um dicionário com os dados de entrada
        input_data = {
            'Temperatura': [temp_input],
            'Precipitação': [precip_input],
            'Umidade': [umidade_input],
            # Variáveis dummies para Fertilizante
            'Fertilizante_Orgânico': [1 if fertilizante_input
== 'Orgânico' else 0],
            'Fertilizante_Sintético':          [1          if
fertilizante_input == 'Sintético' else 0],
            # Variáveis dummies para Tipo de Solo
            'Tipo  de  Solo_Arenoso':  [1  if  solo_input  ==
'Arenoso' else 0],
            'Tipo  de  Solo_Argiloso':  [1  if  solo_input  ==
'Argiloso' else 0],
            'Tipo  de  Solo_Siltoso':  [1  if  solo_input  ==
'Siltoso' else 0],
        }

        # Converter o dicionário em um DataFrame
        input_df = pd.DataFrame(input_data)

        # Garantir que todas as colunas necessárias estejam
presentes
        for col in X.columns:
            if col not in input_df.columns:
                input_df[col]  =  0   # Adicionar  coluna  com
valor zero

        # Reordenar as colunas para corresponder ao conjunto
de treinamento
        input_df = input_df[X.columns]

        # ----------------------------------
        # 4.3. Realizar a Previsão
        # ----------------------------------

        # Fazer a previsão com o modelo treinado
        prediction = model.predict(input_df)

        # Exibir o resultado da previsão
        st.subheader('Resultado da Previsão')
        st.write(f'**Previsão          de          Produção:**
{prediction[0]:.2f} ton/ha')
```

Código-fonte 15 - Novas predições
Fonte: Elaborado pelo autor (2024)

No resultado da nossa aplicação, o usuário insere novos dados, e logo abaixo é exibido o resultado da predição com base no modelo treinado.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

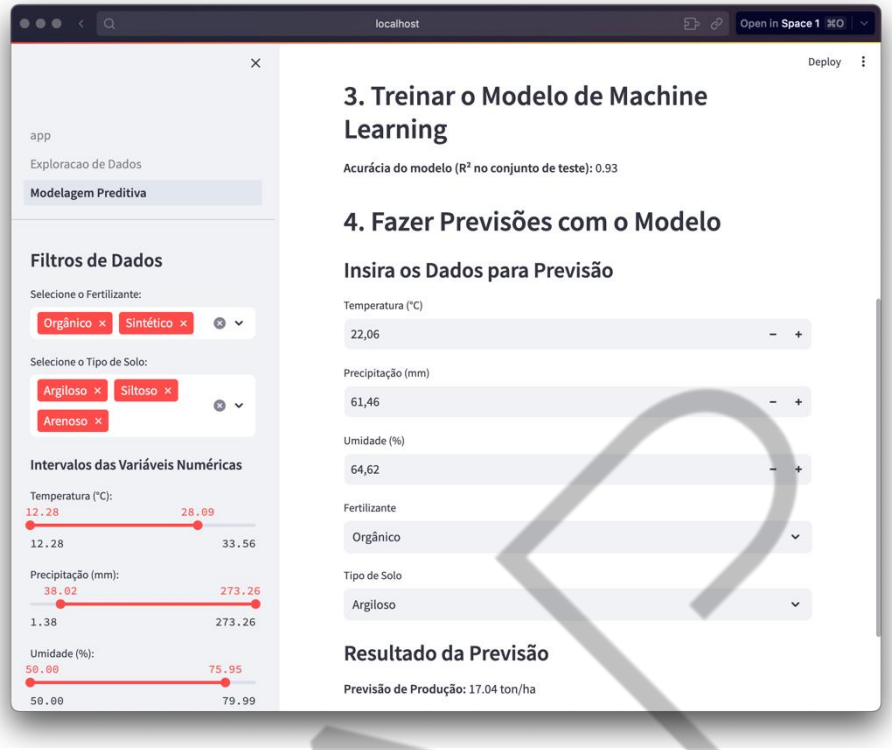


Figura 14 - Novas predições do modelo
Fonte: Elaborado pelo autor (2024)

Após criar uma aplicação completa de Ciência de Dados e Machine Learning utilizando o Streamlit, podemos avançar ainda mais aplicando técnicas de pré-processamento nos dados para aperfeiçoar o treinamento do modelo. Além disso, podemos experimentar e comparar o desempenho de diferentes algoritmos de regressão.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

4 DEPLOY E COMPARTILHAMENTO

Após desenvolvermos nossa aplicação de Ciência de Dados e Machine Learning utilizando o **Streamlit**, o próximo passo é compartilhar essa ferramenta com outras pessoas. O **Streamlit Cloud** é uma plataforma que permite hospedar e compartilhar suas aplicações de forma simples e gratuita.

4.1 Compartilhando aplicações com o Streamlit Cloud

O Streamlit Cloud é um serviço que permite implantar suas aplicações Streamlit na nuvem com apenas alguns cliques. Ele é integrado com repositórios do GitHub, facilitando o processo de deploy.

Benefícios do Streamlit Cloud:

- **Implantação simples:** não é necessário configurar servidores ou ambientes complexos.
- **Atualizações automáticas:** qualquer alteração no seu repositório GitHub é refletida automaticamente na aplicação.
- **Colaboração facilitada:** compartilhe sua aplicação com colegas, clientes ou o público em geral através de um link.

FIAP

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

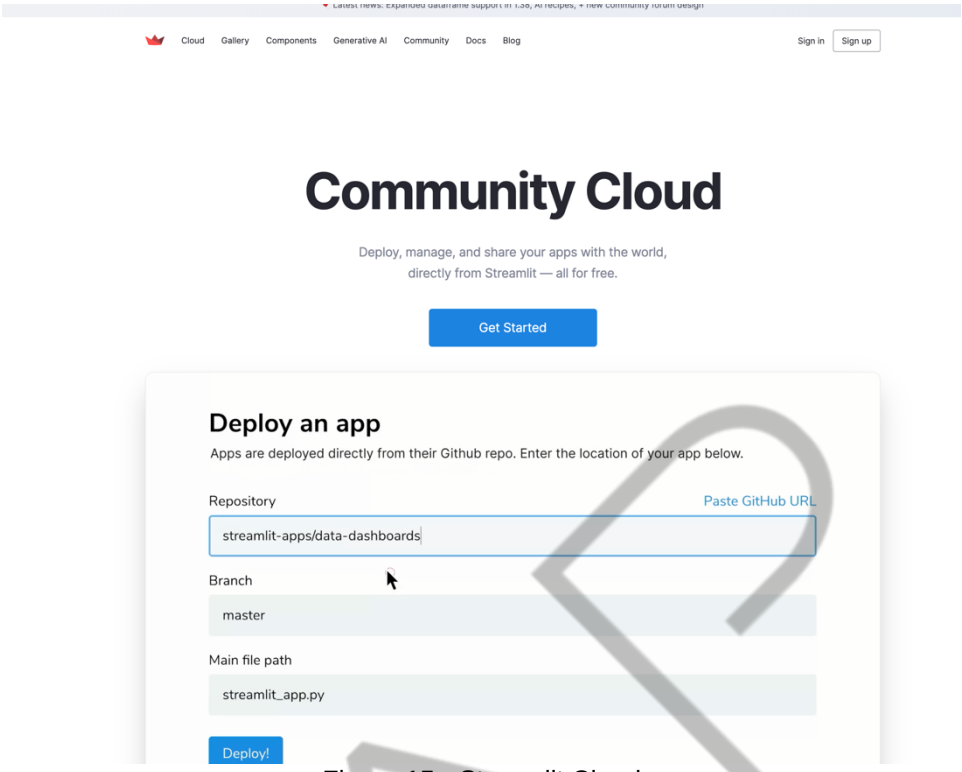


Figura 15 - Streamlit Cloud
Fonte: Streamlit (2024)

4.2 Publicando sua aplicação

Para publicar no Streamlit Cloud é bem simples, e precisamos apenas seguir os passos:

1. **Arquivo requirements.txt:** crie um arquivo chamado requirements.txt na raiz do seu repositório. Este arquivo deve listar todas as bibliotecas utilizadas pela sua aplicação. No nosso caso, usamos: Streamlit, Pandas, Numpy, Scikit-learn, Seaborn, Plotly e Matplotlib.
2. **Criar um Repositório:** crie um novo repositório e faça o upload de todos os arquivos do seu projeto.
3. **Criar uma Conta no Streamlit Cloud:** acesse streamlit.io/cloud e clique em "Sign in with GitHub" para criar uma conta usando seu perfil do GitHub.
4. **Criar a Aplicação:** faça login no Streamlit Cloud, clique em "New app" para iniciar o processo de criação do app. Escolha o repositório, Branch e especifique o caminho para o arquivo principal. No nosso caso, é app.py.
5. Clique em "Deploy" e aguarde alguns minutos.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

6. Acesse o link fornecido pelo Streamlit Cloud para testar sua aplicação.

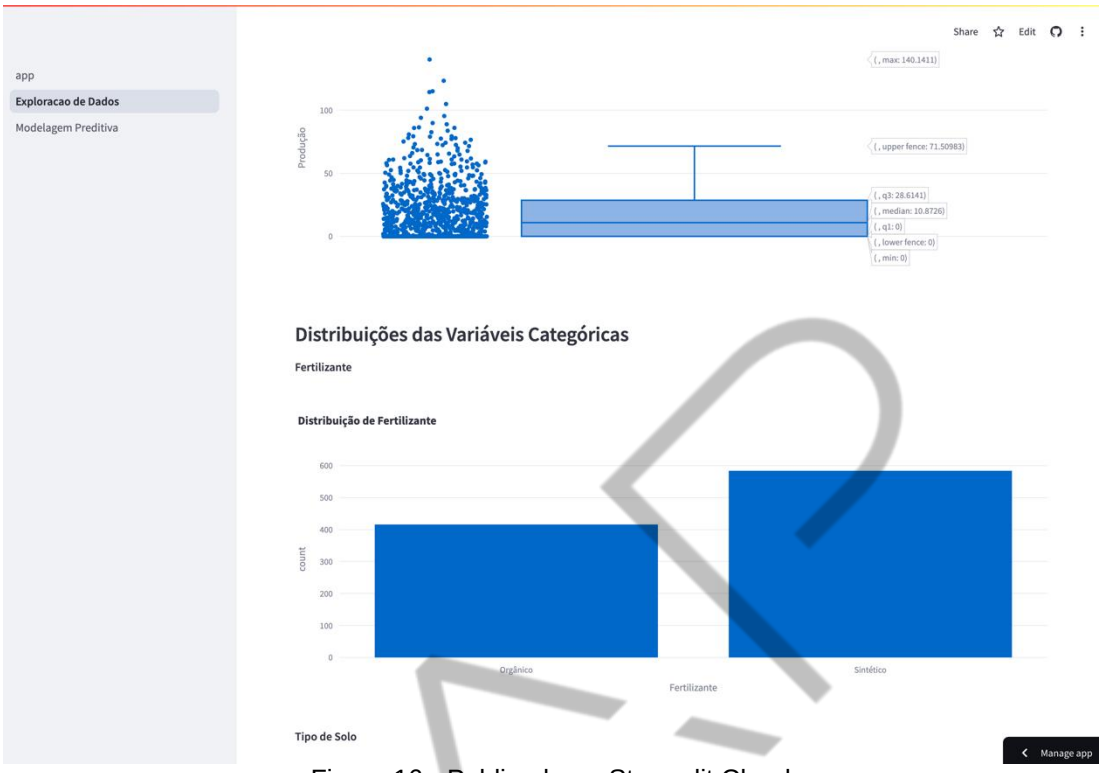


Figura 16 - Publicado no Streamlit Cloud
Fonte: Elaborado pelo autor (2024)

Dica:

- Acesse o GitHub como código completo desta aplicação no link: <https://github.com/arnaldojr/streamlit-demo-dados>.

- Acesse aplicação publicada no Streamlit Cloud no link: <https://fiap-demo.streamlit.app/>.

Publicar nossa aplicação com o Streamlit Cloud é uma maneira rápida e eficaz de compartilhar o projeto com o mundo. Com apenas alguns passos, você pode transformar seus scripts de análise de dados em aplicações Web interativas acessíveis a qualquer pessoa com um link.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

CONCLUSÃO

O Streamlit é uma boa biblioteca para construir uma aplicação Web completa de Ciência de Dados e Machine Learning, desde a exploração inicial dos dados até o treinamento e avaliação de modelos preditivos. Com uma interface interativa e intuitiva, o Streamlit simplifica o processo de transformar scripts de análise em aplicações Web robustas, permitindo que desenvolvedores e cientistas de dados possam interagir com os dados de maneira prática e eficaz.

A integração do Streamlit com bibliotecas populares da área de dados e Machine Learning permite criar um fluxo de trabalho eficiente. Implementamos filtros dinâmicos, visualizações interativas e funcionalidades de previsão que tornaram a aplicação mais rica e informativa. Essa interatividade permite uma exploração mais profunda dos dados, ajudando a extrair *insights* valiosos e a testar diferentes hipóteses rapidamente.

A capacidade do Streamlit de se integrar facilmente com repositórios do GitHub e de ser implantado no Streamlit Cloud facilita o compartilhamento das aplicações com colegas, clientes e a comunidade. O processo de deploy é simples e direto, transformando o código local em uma aplicação Web acessível a partir de qualquer navegador com apenas alguns cliques.

No entanto, é importante destacar algumas desvantagens ao utilizar o Streamlit. A principal limitação é a complexidade na criação de aplicações Web mais avançadas, com funcionalidades de *front-end* customizadas e autenticação de usuários. O Streamlit, apesar de sua simplicidade, não oferece a flexibilidade de *frameworks* web completos, como Flask ou Django, o que pode ser um obstáculo para projetos que demandam funcionalidades específicas. Além disso, a capacidade de escalabilidade e o desempenho em aplicações que exigem alta carga computacional ou integração com serviços externos em tempo real também podem ser limitados, especialmente na versão gratuita do Streamlit Cloud.

FIAP

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

REFERÊNCIAS

SCIKIT-LEARN. **scikit-learn.org**, 2024. Disponível em: <https://scikit-learn.org/stable/>. Acesso em: 01 out. 2024.

STREAMLIT. **streamlit.io**, 2024. Disponível em: <https://streamlit.io/>. Acesso em: 01 out. 2024.

VANDERPLAS, Jake. Python Data Science Handbook: Essential Tools for Working with Data. **Github**, 2016. Disponível em: <https://jakevdp.github.io/PythonDataScienceHandbook/>. Acesso em: 01 out. 2024.

WASKOM, M. L. Seaborn: Statistical Data Visualization. **Journal of Open Source Software**, 60, 2021, p. 3021-3025.

FIAP

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com